Mock Paper 2B — Algorithms & Programming (H446/02) — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

> Marking guidance: award marks for valid alternative wording that conveys the same point. Code answers may be in OCR Exam Reference Language or any high-level language; award marks for correct logic regardless of syntax dialect. Trace marks require working to be shown, not just a final answer.

Question 1 — Computational thinking [11]

(a) [3]: - Technique = **abstraction** (specifically representational abstraction / modelling). **[1]** - The warehouse is represented as a grid of free/blocked cells, removing irrelevant detail (shelf height, robot colour). **[1]** - Why helpful: it produces a simpler model that is sufficient to plan collision-free routes and reduces the amount of data/processing, making path-finding feasible in real time. **[1]**

(b) [4] — two valid sub-problems, each with a sensible input and output (2 marks each): - e.g. *Allocate an order to a robot*: input = order details / item locations + robot statuses; output = chosen robot ID. **[2]** - e.g. *Plan a route for the robot*: input = robot start cell + target shelf cell + grid; output = sequence of cells / moves. **[2]** - (Other valid: locate the item on the shelf; check item availability; route robot to packing station; detect/avoid collisions.)

(c) [2]: - Concurrency must be managed because multiple robots act on / compete for the same shared resource (a cell) at the same time. **[1]** - Consequence if unmanaged: a **collision** / both robots try to occupy the same cell — a race condition leading to deadlock, damage or an inconsistent floor state. **[1]**

(d) [2] — any one well-explained reason for 2, or 1+1: - Categorising jobs lets the controller apply the **same handling rule to a whole class** of jobs (e.g. always service urgent before standard) **[1]**, giving predictable, consistent prioritisation without examining every job individually, which helps guarantee response times. **[1]**

Question 2 — Programming techniques [12]

(a) [1] — e.g. an iterative solution uses a loop and fixed memory; a recursive solution calls itself and uses the call stack (extra memory per call). **[1]** (any one clear difference)

(b) [5] — Output, in order:

```
[15, 25, 35]
3
```

- `nums` is passed **by reference**, so `adjust` modifies the original array; each element gains 5 → `[15, 25, 35]`. **[1]** (effect on array) **+ [1]** (correct printed array)
- `step` is passed **by value**, so the value 5 is used inside the subroutine; nothing about a copy affects the result here but the value added is 5. **[1]**
- `count` is a **global** variable, in scope inside the procedure; it is incremented once per element (3 elements) → prints `3`. **[1]**
- Correct ordering of both printed lines. **[1]**

(c) [3]: - Scope = the region of a program in which a variable is **accessible / has meaning / exists**. **[1]** - (Local = within the subroutine declared; global = whole program.) **[1]** - Disadvantage of a global: any part of the program can change it, causing **unintended side-effects / hard-to-trace bugs**, and it reduces reusability of subroutines / risks name clashes. **[1]** (any one)

(d) [3]:

```
total = 0
k = 1
while k <= 5
    total = total + k
    k = k + 1
endwhile
```

- Correct initialisation of `total` and `k` before the loop. **[1]**
- Correct loop condition `while k <= 5`. **[1]**
- Body adds `k` to `total` **and** increments `k` inside the loop (so it terminates). **[1]**

Question 3 — Recursion and the call stack [10]

(a) [2]: - Base case = `if exp == 0 then return 1` (stops the recursion). **[1]** - Recursive/general case = `return base * power(base, exp - 1)` (calls itself with `exp` reduced, moving towards the base case). **[1]**

(b) [5] — `power(3, 4)`. Frames pushed (each waits for the one below):

```
power(3,4) -> 3 * power(3,3)
power(3,3) -> 3 * power(3,2)
power(3,2) -> 3 * power(3,1)
power(3,1) -> 3 * power(3,0)
power(3,0) -> base case, returns 1
```

Unwinding (popping):

Frame	Evaluates	Returns
power(3,4)	3 * 27	**81**
power(3,3)	3 * 9	27
power(3,2)	3 * 3	9
power(3,1)	3 * 1	3
power(3,0)	base case	1

Marks: - Correct chain of calls pushed: exp = 4 → 3 → 2 → 1 → 0. **[2]** - Base case `power(3,0) = 1` identified. **[1]** - Correct returns on unwinding: 1 → 3 → 9 → 27. **[1]** - Final frame returns 3 × 27 = 81. **[1]**

(c) [1] — `power(3, 4) = 81`. **[1]**

(d) [2]: - Without the base case, `exp` keeps decreasing (4, 3, 2, 1, 0, −1, −2 …) and the function calls itself forever / never returns. **[1]** - The call stack keeps growing until memory is exhausted: a **stack overflow** error. **[1]**

Question 4 — Object-oriented programming [14]

(a) [5] — e.g.:

```
class Robot
    private id
    private batteryLevel
    private carrying

    public procedure new(robotID)
        id = robotID
        batteryLevel = 100
        carrying = false
    endprocedure
endclass
```

- Class declared with name `Robot`. **[1]**
- Three attributes declared `private`. **[1]**
- Constructor takes an `id` parameter and assigns it to the attribute. **[1]**
- Constructor sets `batteryLevel = 100`. **[1]**
- Constructor sets `carrying = false`. **[1]**

(b) [5]:

```
public function pickUp()
    if carrying == false then
        carrying = true
        return true
    else
        return false
    endif
endfunction

public procedure drain(amount)
    batteryLevel = batteryLevel - amount
    if batteryLevel < 0 then
        batteryLevel = 0
    endif
endprocedure
```

- `pickUp` checks `carrying == false`. **[1]**
- `pickUp` sets `carrying = true` and returns `true` when not carrying. **[1]**
- `pickUp` returns `false` when already carrying. **[1]**
- `drain` subtracts `amount` from `batteryLevel`. **[1]**
- `drain` clamps `batteryLevel` so it is never below 0. **[1]**

(c) [2]:

```
class ChargingRobot inherits Robot
    public procedure recharge()
        batteryLevel = 100
    endprocedure
endclass
```

- Correct inheritance syntax (`inherits Robot` / `extends`). **[1]**
- Adds a `recharge()` method that sets `batteryLevel = 100`. **[1]** *(Accept use of a setter / `super` if `batteryLevel` is treated as needing access via the parent.)*

(d) [2]: - Relationship = **inheritance** ("is-a": a ChargingRobot *is a* Robot). **[1]** - Advantage: code reuse — `ChargingRobot` inherits all the attributes/methods of `Robot` without rewriting them / changes to shared behaviour need only be made once. **[1]** (any one)

Question 5 — Computational methods [11]

(a) [3]: - A problem is **solvable but intractable** if an algorithm exists to solve it exactly, but the time/resources required grow so fast (e.g. exponentially/factorially) that it cannot be solved in a reasonable time for large inputs. **[1]** (idea of solvable) **+ [1]** (idea of not in reasonable time / explosive growth) - Example: the **Travelling Salesman Problem** / optimal scheduling / optimal multi-robot routing visiting all points. **[1]**

(b) [3]: - Optimal algorithm guarantees the best possible solution but may be slow/exhaustive. **[1]** - Heuristic uses a "rule of thumb" to find a **good-enough** solution quickly without guaranteeing optimality.

[1] - Reason preferred for GridStore: routes must be planned in **real time** for many robots; a near-optimal route found instantly is more useful than a perfect route found too late. **[1]**

(c) [1]: - **Constraint-based / constraint satisfaction** problem solving (allow "backtracking" as the technique used to solve it). **[1]**

(d) [2]: - Divide and conquer = break a problem into smaller sub-problems, solve each (often recursively), then **combine** the results. **[1]** - Example (non-sorting): **binary search** (or the recursive nearest-pair, fast Fourier transform, etc.). **[1]**

(e) [2]: - Benefit of caching: a previously planned route between common cells can be **reused instantly**, saving repeated computation / reducing latency. **[1]** - Becomes invalid when: the grid changes — e.g. a shelf/robot now blocks a cell on the cached route, or a cell becomes congested — so the stored route is no longer free/valid. **[1]**

Question 6 — Big O analysis [11]

(a) [2]: - Big O describes how an algorithm's **run-time (or space) requirement grows as the input size *n* grows** / its order of growth in the worst case. **[1]** - It uses *n* rather than seconds because actual time depends on hardware/implementation; Big O captures the **rate of growth**, which is what matters for scalability and lets algorithms be compared independently of the machine. **[1]**

(b) [4] — 1 mark each: - (i) Linear search (unsorted): **O(n)**. **[1]** - (ii) Binary search: **O(log n)**. **[1]** - (iii) Merge sort: **O(n log n)**. **[1]** - (iv) Array access by index: **O(1)**. **[1]**

(c) [3]: - Complexity = **O(log n)**. **[1]** - Justification: `i` is **halved** each iteration (`i = i DIV 2`), so the number of iterations is roughly $\log_2 n$ before `i` reaches 1. **[1]** - The loop body is constant work, so the total grows logarithmically with n. **[1]**

(d) [2]: - **Y ($O(2^n)$)** scales worse. **[1]** - Exponential growth (2^n) eventually overtakes any polynomial (n^2) and grows far faster as n increases — e.g. each extra input item doubles the work for Y, whereas X grows only quadratically. **[1]**

Question 7 — Searching algorithms [10]

(a) [1] — The list must be **sorted** (in order). **[1]** (Accept: random/direct access to elements by index is available.)

(b) [5] — Searching for 16 in `[4,9,16,23,31,38,52,61,77]` (indices 0–8), `mid = (low+high) DIV 2`:

Step	low	high	mid	list[mid]	Action
1	0	8	4	31	16 < 31 → search left, high = 3
2	0	3	1	9	16 > 9 → search right, low = 2
3	2	3	2	16	**found**

Marks: - Step 1 correct (mid = 4, list[mid] = 31, go left, high = 3). **[1]** - Step 2 correct (mid = 1, list[mid] = 9, go right, low = 2). **[1]** - Step 3 correct (mid = 2, list[mid] = 16, found). **[1]** - `mid` calculated correctly each step using integer division. **[1]** - Number of comparisons of `list[mid]` = **3**. **[1]**

(c) [4]:

```
function binarySearch(list, target)
    low = 0
    high = list.length - 1
    while low <= high
        mid = (low + high) DIV 2
        if list[mid] == target then
            return mid
        elseif list[mid] < target then
            low = mid + 1
        else
            high = mid - 1
        endif
    endwhile
    return -1
endfunction
```

- `if list[mid] == target then return mid`. **[1]**
- `list[mid] < target` branch sets `low = mid + 1`. **[1]**
- else branch sets `high = mid - 1`. **[1]**
- `return -1` after the loop (not found). **[1]**

Question 8 — Sorting algorithms [13]

(a) [5] — Bubble sort of `[6, 3, 8, 2, 5]`, full list after each complete pass:

Pass	List state after pass	Swaps?
1	`[3, 6, 2, 5, 8]`	yes
2	`[3, 2, 5, 6, 8]`	yes
3	`[2, 3, 5, 6, 8]`	yes
4	`[2, 3, 5, 6, 8]`	**no swaps**

- `[3, 6, 2, 5, 8]` after pass 1. **[1]**
- `[3, 2, 5, 6, 8]` after pass 2. **[1]**
- `[2, 3, 5, 6, 8]` after pass 3 (now sorted). **[1]**
- Pass 4 makes no swaps. **[1]**
- States that sorting can first be detected (no swaps) on **pass 4**. **[1]** *(Accept detection on pass 3 if the candidate's implementation tracks "no swap" within the same pass that produces the sorted order; award the mark for a consistent, correctly-justified pass number.)*

(b) [3]: - Worst case: **O(n²)**. **[1]** - Best case: **O(n)** (with swap-detection). **[1]** - Best case occurs when the input is **already sorted** — one pass with no swaps detects completion. **[1]**

(c) [3] — Quick sort of `[7, 2, 9, 4, 3, 8, 1]`, pivot = last element of each (sub)list.

- Partition whole list, pivot = **1**: everything ≥ 1 goes right → `[1 | 2, 9, 4, 3, 8, 7]`. Left = `[]`, right = `[2, 9, 4, 3, 8, 7]`. **[1]**

- Partition `[2, 9, 4, 3, 8, 7]`, pivot = **7** → `[2, 4, 3 | 7 | 9, 8]`. Left = `[2, 4, 3]`, right = `[9, 8]`. **[1]**
- Partition `[2, 4, 3]`, pivot = **3** → `[2 | 3 | 4]`; partition `[9, 8]`, pivot = **8** → `[8 | 9]`. Recombining gives the sorted list **`[1, 2, 3, 4, 7, 8, 9]`**. **[1]**

(Full picture for reference:)

```
[7, 2, 9, 4, 3, 8, 1]           pivot 1 -> 1 | [2,9,4,3,8,7]
           [2, 9, 4, 3, 8, 7]   pivot 7 -> [2,4,3] | 7 | [9,8]
   [2,4,3]                      pivot 3 -> [2] | 3 | [4]
                       [9,8]    pivot 8 -> [8] | 9
Result: [1, 2, 3, 4, 7, 8, 9]
```

(d) [2]: - Worst case occurs when the pivot is consistently the smallest or largest element (e.g. an **already-sorted (or reverse-sorted) list with last-element pivot**), giving maximally unbalanced partitions of size n−1 and 0 → $O(n^2)$. **[1]** - Advantage over merge sort: quick sort sorts **in place**, needing only O(log n) extra stack space rather than O(n) auxiliary memory. **[1]** (Accept: typically faster in practice due to good cache behaviour / lower constant factors.)

Question 9 — Data structure algorithms [12]

(a) [5]:

```
procedure enqueue(value)
    if size == 5 then
        print("Queue full")
    else
        rear = (rear + 1) MOD 5
        q[rear] = value
        size = size + 1
    endif
endprocedure

function dequeue()
    if size == 0 then
        return "Queue empty"
    else
        value = q[front]
        front = (front + 1) MOD 5
        size = size - 1
        return value
    endif
endfunction
```

- `enqueue`: advance `rear` with wrap-around `(rear + 1) MOD 5`. **[1]**
- `enqueue`: store `q[rear] = value` and increment `size`. **[1]**
- `dequeue`: read `value = q[front]` before advancing. **[1]**
- `dequeue`: advance `front` with wrap-around `(front + 1) MOD 5` and decrement `size`. **[1]**

- `dequeue`: returns the dequeued value. **[1]**

(b) [3] — trace from empty (`front = 0`, `rear = -1`, `size = 0`):

Operation	front	rear	size	contents (front→rear)
enqueue(A)	0	0	1	A
enqueue(B)	0	1	2	A, B
enqueue(C)	0	2	3	A, B, C
dequeue()	1	2	2	B, C
enqueue(D)	1	3	3	B, C, D
dequeue()	2	3	2	C, D
enqueue(E)	2	4	3	C, D, E

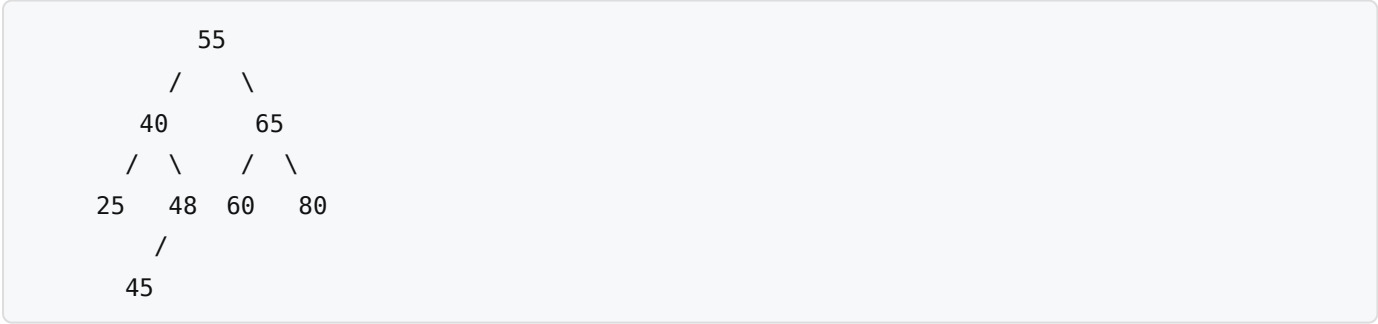
- Final `front = 2`. **[1]**
- Final `rear = 4`. **[1]**
- Contents front→rear = **C, D, E**. **[1]**

(c)(i) [2] — Traverse from head (index 2): - 2 ("Lima") → 3 ("Cairo") → 1 ("Oslo") → 4 ("Quito"). **[1]** - Output order: **Lima, Cairo, Oslo, Quito**. **[1]**

(c)(ii) [2] — Delete "Cairo" (index 3). Node 2 ("Lima") currently points to 3; "Cairo" points to 1: - Node 2 ("Lima") pointer changes from 3 to **1** (bypasses Cairo). **[1]** - New traversal: **Lima, Oslo, Quito**. **[1]**

Question 10 — Trees and traversals [11]

(a) [3] — Insert order `55, 40, 65, 25, 48, 60, 80, 45`:



- Root 55 with 40 (left) and 65 (right). **[1]**
- 25 and 48 as children of 40; 60 and 80 as children of 65. **[1]**
- 45 placed as **left child of 48** (45 > 40 → right to 48; 45 < 48 → left of 48). **[1]**

(b) [3]: - **Pre-order** (root, left, right): `55, 40, 25, 48, 45, 65, 60, 80`. **[1]** - **In-order** (left, root, right): `25, 40, 45, 48, 55, 60, 65, 80`. **[1]** - **Post-order** (left, right, root): `25, 45, 48, 40, 60, 80, 65, 55`. **[1]**

(c) [2]: - A standard in-order traversal visits left, root, right and gives **ascending** order. To get **descending** order, visit **right subtree, then root, then left subtree** (a "reverse in-order" traversal). **[1]** - This outputs the values largest-first directly, with no separate sort needed. **[1]**

(d) [3]: - Best case = **O(log n)**. **[1]** — when the tree is **balanced**, each comparison halves the remaining nodes. **[1]** - Worst case = **O(n)**, when the tree is **unbalanced / degenerate** (effectively a linked list, e.g. from inserting already-sorted data). **[1]**

Question 11 — Shortest path (Dijkstra / A*) [12]

Graph edges (undirected): A-B=4, A-C=3, B-C=1, B-D=2, C-D=5, C-E=8, D-E=2, D-F=6, E-F=3.

(a) [7] — Dijkstra from A. Tentative distances (∞ = unknown); each round finalise the unvisited node with the smallest tentative distance and relax its neighbours.

Finalised this round	**A**	**B**	**C**	**D**	**E**	**F**
start	0	∞	∞	∞	∞	∞
A (0)	0	4 (via A)	3 (via A)	∞	∞	∞
C (3)	0	4 (A) / 4 (via C: 3+1)	3	8 (via C: 3+5)	11 (via C: 3+8)	∞
B (4)	0	4	3	6 (via B: 4+2)	11	∞
D (6)	0	4	3	6	8 (via D: 6+2)	12 (via D: 6+6)
E (8)	0	4	3	6	8	11 (via E: 8+3)
F (11)	0	4	3	6	8	11

Order finalised: **A, C, B, D, E, F.**

- Shortest distance A → F = **11**. **[1]**
- Shortest path = **A** → **B** → **D** → **E** → **F**. **[1]**

(Path note: F's best is via E (8+3=11). E's best is via D (6+2=8). D's best is via B (4+2=6). B's best is via A (4). C, although finalised earlier at 3, does not lie on the shortest path to F because routing through B gives a cheaper D.)

Working marks: - Initialise A = 0, others ∞; relax A's neighbours (B = 4, C = 3). **[1]** - Finalise C (= 3); relax to give D = 8 and E = 11 (B stays 4 since 3+1 = 4 is not smaller). **[1]** - Finalise B (= 4); update D to 6 (4+2). **[1]** - Finalise D (= 6); update E to 8 (6+2) and F to 12 (6+6). **[1]** - Finalise E (= 8); update F to 11 (8+3); correct visiting order A, C, B, D, E, F shown. **[1]**

(Award the distance and path marks (the first two listed) plus the five working marks = 7.)

(b) [3]: - A* orders nodes by **f(n) = g(n) + h(n)**. **[1]** - where **g(n)** = the actual cost of the path found so far from the start to n, and **h(n)** = the heuristic estimate of the remaining cost from n to the goal. **[1]** - Advantage: the heuristic **guides** the search towards the goal, so A* typically expands **fewer nodes** than Dijkstra (which expands uniformly outward), finding the path faster. **[1]**

(c) [2]: - The heuristic must be **admissible** — it must **never overestimate** the true remaining cost to the goal. **[1]** - If it overestimated, A* could discard the genuinely shortest route (treating it as more expensive than it is) and return a **sub-optimal path**. **[1]** (Accept "consistent/monotonic" as the property.)

Question 12 — Design and evaluation [13]

Level-of-response marking (0–13). Award the band that best fits, then a mark within it.

- **Level 3 (10–13):** A well-structured response that addresses **all four** required points. Data structure choice is justified (with the frequent-update requirement considered), the searching approach is appropriate and explained with reference to relevant computational methods/techniques, time complexity is analysed correctly as the number of robots grows, and at least one trade-off is evaluated with a reasoned conclusion. Technically accurate, correct terminology throughout. QWC strong.
- **Level 2 (5–9):** Addresses most points with some justification. Data structure and search approach identified and mostly appropriate; some complexity discussion; a trade-off mentioned but evaluation limited. Mostly accurate, reasonable structure.
- **Level 1 (1–4):** Fragmentary/descriptive. Names a structure and/or search method with little justification; little or no complexity analysis or evaluation. Some inaccuracies.
- **0 marks:** nothing creditworthy.

Indicative content (credit any valid, well-argued combination):

Data structures: - A **hash table / dictionary keyed by robot ID** giving **O(1)** average-case status updates and lookups — important because robots change status many times per second. - A **spatial structure** (uniform grid of buckets keyed by floor cell, a quadtree, or a k-d tree) so "nearest free robot" queries only examine robots near the pickup cell rather than all robots. - A **priority queue / min-heap** to retrieve robots ordered by distance/ETA, or to drive Dijkstra/A *expansion. - Optionally model the warehouse floor as a weighted graph** of cells for true travel-distance nearest-robot queries.

Searching / algorithmic approach: - For straight-line nearest: query the spatial index for candidate robots in the pickup cell and adjacent cells, filter to those that are **free and have sufficient battery**, then take the minimum distance (a localised linear scan of few candidates). - For travel-distance nearest on the grid: **Dijkstra / A*** from the pickup cell; A* with an admissible straight-line (Manhattan/Euclidean) heuristic explores fewer cells. - A **heuristic** (e.g. straight-line distance, ignoring obstacles) may be "good enough" for real-time selection rather than computing exact path length to every robot.

Time complexity: - Naïve scan of all robots = **O(n)** per order → with thousands of orders/min and many robots this becomes a bottleneck. - A spatial index reduces each query to robots in nearby cells, ~**O(k)** with $k \ll n$, or **O(log n)** for tree structures — far more scalable as n grows. - Status updates should be **O(1)** (hash table) or O(log n) so very frequent changes don't dominate.

Trade-offs to evaluate (need at least one, with a judgement): - **Speed vs accuracy:** a straight-line heuristic is fast but ignores blocked cells/congestion; exact path-finding is accurate but costly. - **Speed vs memory:** spatial indexes/caches consume extra memory to gain query speed. - **Responsiveness vs optimality:** in real time, dispatching a near-optimal robot instantly beats finding the perfect robot too late. - **Concurrency:** updates and queries happen simultaneously — must avoid race conditions when marking a robot as taken (links to Q1c).

A response reaching Level 3 makes a clear recommendation (e.g. hash table keyed by robot ID for O(1) status + spatial grid for candidate selection + A* for travel-distance ranking) and justifies it against these trade-offs.

Verified mark total

Question	Marks
Q1 Computational thinking	11
Q2 Programming techniques	12
Q3 Recursion & call stack	10
Q4 Object-oriented programming	14
Q5 Computational methods	11
Q6 Big O analysis	11
Q7 Searching algorithms	10
Q8 Sorting algorithms	13
Q9 Data structure algorithms	12
Q10 Trees and traversals	11
Q11 Shortest path (Dijkstra / A*)	12
Q12 Design and evaluation	13
TOTAL	**140**

Verified total = 140 marks.